

2). Repeat this attack, but instead of tampering with Y_A and Y_B , tamper with the generator α . Show that Mallory can recover the messages m_0 and m_1 from their ciphertexts by setting α to 1, q , or $q-1$.

Task 3: Implement “textbook” RSA & MITM Key Fixing via Malleability: 1). RSA has two core components: key generation and encryption/decryption. (90% of the work is in implementing key generation.) Your implementation should support variable length primes (up to 2048 bits), and use the value $e=65537$. Feel free to use your cryptographic library’s interface for generating large primes but implement the rest—including computing the multiplicative inverse - yourself.

Encrypt and decrypt a few messages to yourself to make sure it works. Remember messages must be integers in Z_n^* (that is, less than n , the product of the two primes). You can convert an ASCII string to hex and then turn that hex value into an integer.

2). From 1). you just implemented “textbook” RSA, and it is widely insecure. Because it is too slow and inconvenient to operate on a large amount of data directly, RSA is often used to exchange a symmetric key that will be used to encrypt future messages. It would be terrible, of course, if an adversary were able to learn that key. And that’s what we’re about to do. One of textbook RSA’s great weaknesses is its malleability, i.e. an active attacker can change the meaning of the plaintext message by performing an operation on the respective ciphertext. To demonstrate the dangers of malleability, implement the following protocol:

Alice		Mallory	Bob	
Computes	Sends	Modifies	Compute s	Sends
	n,e			
			$s \in \mathbb{Z}_n^* c = s^e \text{ mod } n$	
				c
		$c \rightarrow c'$ (Mallory sends c' to Alice instead of c)		
$s = c'^d \text{ mod } n$				
$k = \text{SHA256}(s)$				
$m = \text{"Hi Bob!" } c_0 = \text{AES-CBC}_k(m)$	c_0			

Find a value for c' that Mallory will substitute for the ciphertext c that will allow Mallory to decrypt the ciphertext c_0 and recover m . Hint: Mallory knows Alice's public key (n,e) and can send a value of c' to Alice that will allow Mallory to know the value of s without knowing Alice's private key. Give another example of how RSA's malleability could be used to exploit a system (e.g. to cause confusion, disruption, or violate integrity).

Malleability can also affect signature schemes based on RSA. Consider the scheme:

$$\text{Sign}(m,d) = m^d \bmod n$$

Suppose Mallory sees the signatures for two messages m_1 and m_2 . Show how Mallory can create a valid signature for a third message, $m_3 = m_1 \cdot m_2$.

Alice		Bob	
Privately Computes	Sends	Privately Computes	Sends
	q, α		
$X_A \in \mathbb{Z}_q \ Y_A = \alpha^{(X_A)} \bmod q$		$X_B \in \mathbb{Z}_q \ Y_B = \alpha^{(X_B)} \bmod q$	
	Y_A		Y_B
$s = (Y_B)^{(X_A)} \bmod q$ Forget X_A		$s = (Y_A)^{(X_B)} \bmod q$ Forget X_B	
$k = \text{SHA256}(s)$		$k = \text{SHA256}(s)$	
$m_0 = \text{"Hi Bob!" } c_0$ $= \text{AES-CBC}_k(m_0)$	c_0	$m_1 = \text{"Hi Alice!" } c_1$ $= \text{AES-CBC}_k(m_1)$	c_1

Protocol 1: A Diffie-Hellman Key-agreement